

Following one load instruction

A waveform guide for the RV32I multicycle core

Example instruction

```
lw x5, 12(x3)
```

The instruction adds 12 to x3, reads a 32-bit word from that address, and writes the word into x5.

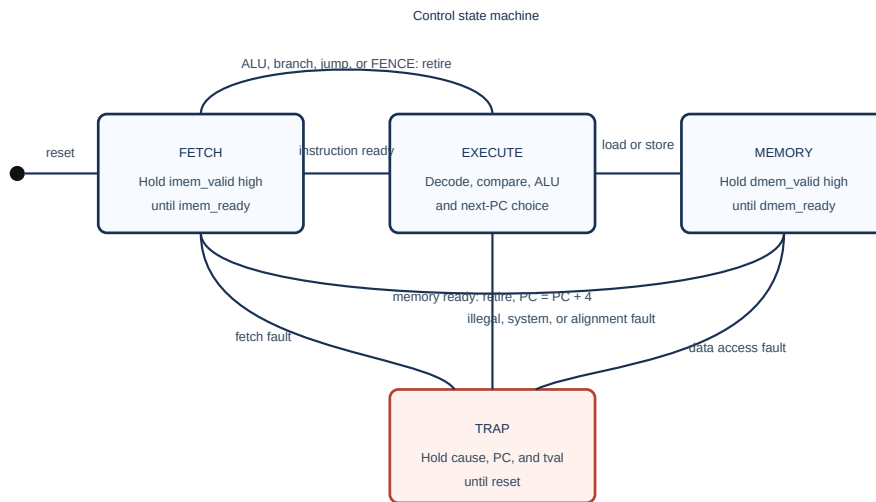


Figure 1. The path taken by LW is FETCH, EXECUTE, MEMORY, FETCH.

Signals to place in the waveform

Signal	What to watch
<code>debug_state_o</code>	FETCH, EXECUTE, MEMORY, then FETCH
<code>imem_valid_o</code> / <code>imem_ready_i</code>	Instruction request and acceptance
<code>dmem_valid_o</code> / <code>dmem_ready_i</code>	Load request and completion
<code>dmem_addr_o</code>	The value of x3 plus 12
<code>retire_valid_o</code>	One-cycle pulse when LW finishes

1. Fetch

`debug_state_o` is `STATE_FETCH`. `imem_addr_o` equals the current PC and `imem_valid_o` is high. If instruction memory is slow, the address does not change.

When `imem_ready_i` becomes high, the core saves `imem_rdata_i` in `instruction_q`. The PC has not advanced yet. Keeping the PC on the current instruction makes trap reporting and PC-relative arithmetic straightforward.

2. Decode and address generation

The state changes to `STATE_EXECUTE`. Opcode `0000011` selects the load group. `funct3=010` selects a signed 32-bit word, which is `LW`.

The decoder requests an I-type immediate, a register write, a memory read, and the memory writeback path. The immediate generator takes instruction bits 31 through 20 and sign extends them. For this instruction its output is 12.

The register file reads `x3`. The ALU adds `x3` and 12. If the low two result bits are not zero, the core records a load-address-misaligned trap and never sends a memory request.

Expected execute values

Item	Value
Source register	<code>rs1 = x3</code>
Immediate	12
ALU operation	<code>x3 + 12</code>
Destination	<code>rd = x5</code>
Access size	32-bit word

3. Memory wait

For an aligned address, the state changes to `STATE_MEMORY`. `dmem_addr_o` is the ALU sum, `dmem_valid_o` stays high, and `dmem_write_o` is zero.

Nothing retires while `dmem_ready_i` is low. The core is waiting, not repeating the instruction. When ready rises, `dmem_error_i` causes a load access fault. Otherwise the 32-bit `dmem_rdata_i` value is written to `x5` on the clock edge.

Handshake timeline

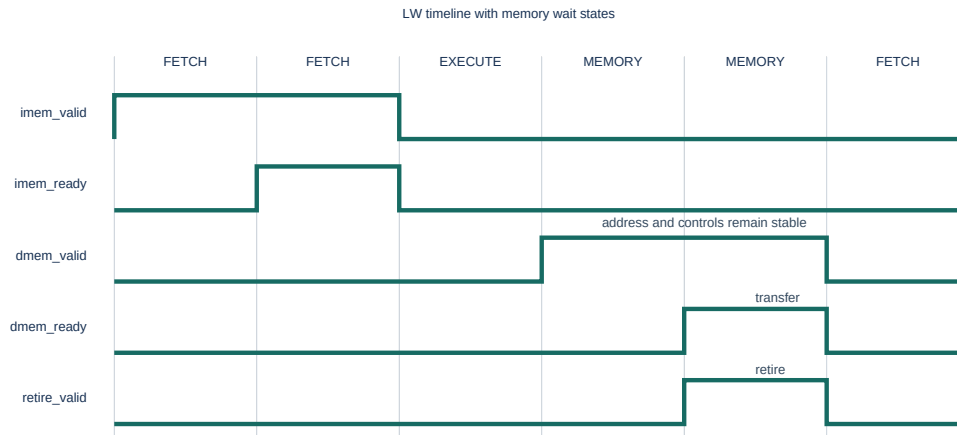


Figure 2. A load request remains stable until ready completes it.

4. Retirement

The PC increases by four, `retire_valid_o` pulses, and the state returns to fetch. The register-file write and retirement pulse happen on the same completion edge.

Try the byte-load variation

```
lb x5, 13(x3)
```

The byte address is legal even when its low bits are nonzero. Memory still returns the aligned 32-bit word. The load-align block selects one byte using the low address bits and sign extends bit 7 before writing `x5`.

Useful comparison: change LB to LBU and use a returned byte whose high bit is one. LB fills the upper 24 bits with ones; LBU fills them with zeros.

Reproduce the waveform

Run `make test-cocotb-waves PYTHON=.venv/bin/python`. Verilator writes `sim/build/cocotb/dump.fst`; open it with the checked-in `waves/rv32i_core.gtkw` signal grouping. The cocotb test adds zero-to-three-cycle memory delays so stable request intervals are visible.

GTKWave is a viewer, not a verification oracle. The pass/fail result comes from the cocotb assertions and retirement comparison; the FST trace supports diagnosis and teaching.